

From Arad to Bucharest (DLS, $l=4$)

F: Arad (0)

C:

F: Zerind (-1), Sibiu (-1), Timisoara (-1)

C: Arad

F: Arad (-2), Oradea (-2), Sibiu (-1), Timisoara (-1)

C: Arad, Zerind

F: Oradea (-2), Sibiu (-1), Timisoara (-1)

C: Arad, Zerind

F: Sibiu (-3), Zerind (-3), Sibiu (-1), Timisoara (-1)

C: Arad, Zerind, Oradea

F: Arad (-4), Fagaras (-4), Oradea(-4), RV (-4), Zerind(-3), Sibiu (-1), Timisoara (-1)

C: Arad, Zerind, Oradea, Sibiu

As $l=4$ (maximum depth) no state in a node at depth == 4 can be expanded.

F: Zerind(-3), Sibiu (-1), Timisoara (-1)

C: Arad, Zerind, Oradea, Sibiu

F: Arad (-2), Lugoj (-2)

C: Arad, Zerind, Oradea, Sibiu, Timisoara

F: Mehadla (-3), Timisoara (-3)

C: Arad, Zerind, Oradea, Sibiu, Timisoara, Lugoj

F: Dobreta(-4), Lugoj(-4), Timisoara (-3)

C: Arad, Zerind, Oradea, Sibiu, Timisoara, Lugoj, Mehadla

As $l=4$ (maximum depth) no state in a node at depth == 4 can be expanded.

F: empty

C: Arad, Zerind, Oradea, Sibiu, Timisoara, Lugoj, Mehadla

NO SOLUTION IS FOUND.

But we can observe that there is a path of length 3 (A, S, F, B) and another of length 4 (A, S, RV, P, B) between Arad and Bucharest.

What has happened?

In the first version of the Graph Search it is executed a boolean test to check if a state belongs to the visited list. We only consider information from the state.

What does it happen if we add information from the node the state belongs to?

Could be considered a node more promising than another one by extracting information from its attributes? In this case as the strategy is DLS the critical attribute is the depth.

From Arad to Bucharest (DLS, l=4)

F: (Arad, 0)

C:

F: (Zerind, -1), (Sibiu, -1), (Timisoara, -1)

C: (Arad, 0)

F: (Arad, -2), (Oradea, -2), (Sibiu, -1), (Timisoara, -1)

C: (Arad, 0), (Zerind, -1)

We extract (Arad, -2) from the Frontier. In the Visited list we have (Arad, 0). (Arad, -2) has a deeper depth than (Arad, 0) So it is worse and it is not expanded and so, the visited list is not modified.

F: (Oradea, -2), (Sibiu, -1), (Timisoara, -1)

C: (Arad, 0), (Zerind, -1)

F: (Sibiu, -3), (Zerind, -3), (Sibiu, -1), (Timisoara, -1)

C: (Arad, 0), (Zerind, -1), (Oradea, -2)

F: (Sibiu, -3), (Sibiu, -1), (Timisoara, -1)

C: (Arad, 0), (Zerind, -1), (Oradea, -2)

F: (Arad, -4), (Fagaras, -4), (Oradea, -4), (RV, -4), (Zerind, -3), (Sibiu, -1), (Timisoara, -1)

C: (Arad, 0), (Zerind, -1), (Oradea, -2), (Sibiu, -3)

As l=4 (maximum depth) no state in a node at depth == 4 can be expanded. As they are not expanded they are not added to the Closed List.

F: (Zerind, -3), (Sibiu, -1), (Timisoara, -1)

C: (Arad, 0), (Zerind, -1), (Oradea, -2), (Sibiu, -3)

(Zerind, -3) is not better than (Zerind, -1), so it is not expanded.

F: (Sibiu, -1), (Timisoara, -1)

C: (Arad, -0), (Zerind, -1), (Oradea, -2), (**Sibiu, -3**)

Now, (Sibiu, -1) is extracted. (Sibiu, -3) is in the visited list, but (Sibiu, -1) is better because it is in a shallower position.

So (Sibiu, -1) is expanded and overwrites (Sibiu, 3) in the closed list.

F: (Arad, -2), (Fagaras, -2), (Oradea, -2), (RV, -2), (Timisoara, -1)

C: (Arad, -0), (Zerind, -1), (Oradea, -2), (**Sibiu, -1**)

(Arad, -2) is not considered. (Fagaras, -2) is expanded.

F: (Bucharest, -3), (Sibiu, -3), (Oradea, -2), (RV, -2), (Timisoara, -1)

C: (Arad, 0), (Zerind, -1), (Oradea, -2), (Sibiu, -1), (Fagaras, -2),

(Bucharest, -3) is extracted and it satisfies the Goal Test and then the algorithm finishes.

Then the solution is the sequence of actions:

(Go To Sibiu, Go To Fagaras, Go To Bucharest)

Conclusions:

In the first approach (Sibiu, -1) was not expanded. In this branch appeared Bucharest at depth=3, but as it was pruned, No solution was found.

So, in DFS and its variants, if a state is extracted from the frontier and it belongs to a more promising node (shallower) **it must be expanded**.

What does it happen with another strategies?

In BFS, a repeated state always appears associated to a deeper node. So, in this case the boolean control of visited states works.

In UCS, the critical attribute of a node is the path cost. No state can appear in a repeated way associated to a node with a smaller cost. So, boolean works too.

Greedy, it happens the same than in the previous one.

A*, it can appear a state associated with a more promising node as in DLS.

So, to unify:

And the code of the search algorithm where:

if state[node] not in closed or node is **more promising then**

Then, the visited list should contain a pair:

BFS (state, value)

DFS (state, value) But, for **DFS and its variants the value is negative**.

UCS (state, value)

So for BFS, DFS, UCS (**state, abs(value)**) is valid for all of them and now, we have to represent in the code the "node is more promising".